

CSEN 296B-2 | Week 7.2 Lab Submission

AstraNotes Testing Strategy and First Test Set

Student Name: Atishay Jain | **Date:** May 17, 2026 | **Project:** AstraNotes | **Technical Path:** Python

Testing Strategy

AstraNotes is a 3-tier app: a thin presentation tier, a logic tier (*NoteManager*, *ValidationLayer*, *PrivacyService*), and a data tier (*JsonFileRepository*). My testing money goes where the risk is, and the risk is not in the presentation tier - it is in the two places where a bug is silent and expensive: a note that fails to persist, and a private note whose body leaks to disk in plaintext. So the strategy is **test the logic and data tiers hard and first, test the presentation tier only at the seam, and do not chase coverage where a failure would be loud and obvious anyway.**

1. Trace before you test. Every test names the requirement or user story it defends. An untraceable test is noise and gets cut - the same Working Agreement rule that governs feature work.

2. Test behavior at the right level. A title rule is a unit concern. 'A private note is unreadable on disk' is an integration concern. 'Create a note then see it listed' is a feature concern.

3. Realistic first choices. I am not testing everything in week one. Performance (NFR-01/03), the GUI widget tree, search (US-06, not built yet), and FR-08 sort edge cases are deliberately out of the first set, with reasons given at the end.

Two Features Tested First

I picked the two features whose failure modes are the most damaging and the least visible.

Feature A - Create a note (US-01; FR-01, FR-06, SPR-02)

Creating a note is the entry point for every other story; if create is wrong, everything downstream is built on sand. The subtle part is the validation gate: a whitespace-only title must be rejected *before* anything touches disk, and the rejection must be a domain error, not a raw traceback.

Feature B - Private note encrypted at rest (US-04; FR-04, SPR-01, SPR-02)

This is the highest-risk behavior in the whole system. If it silently fails, a user's private note is written to disk in plaintext and nobody notices until it is too late. It is also the one behavior that *only* exists when three components are wired together (manager - privacy - repository), so it cannot be proven by any single unit test.

First Test Set

These are real tests in the suite today (34 passing), grouped by the level each one belongs to. I present the actual set rather than an outline because the code exists and the set is small and focused on purpose.

Feature A - Create a note

Level	Test	What it pins	Traces to
Unit	test_valid_title_passes	Non-empty title passes the gate	FR-01
Unit	test_empty_title_raises	Empty title to ValidationError	FR-01, SPR-02
Unit	test_whitespace_only_title_raises	Whitespace title stripped then rejected	FR-01
Unit	test_note_timestamps_default_to_utc_now	created_at/modified_at UTC-aware on construction	FR-06
Unit	test_note_ids_are_unique	Each note gets a distinct UUID	FR-01
Integration	test_create_note_persists_plaintext_body	create_note validates then writes one {uuid}.json	FR-01, FR-05
Integration	test_create_note_rejects_whitespace_title	Gate fires through the manager, nothing written	FR-01, SPR-02
Feature	test_create_then_list_round_trip	Created note is visible via the user-facing entry point	US-01
Feature	test_create_rejects_empty_title	User sees a friendly message, not a stack trace	US-01, SPR-02

Feature B - Private note encrypted at rest

Level	Test	What it pins	Traces to
Unit	test_encrypt_decrypt_round_trip	Ciphertext is bytes, differs from plaintext, decrypts back	SPR-01
Unit	test_decrypt_with_wrong_key_raises_persistence_error	Wrong key fails loud, never returns garbled text	FR-04, SPR-02
Integration	test_create_private_note_encrypts_body_on_disk	On-disk JSON has NO plaintext body; base64 ciphertext	SPR-01
Integration	test_list_notes_decrypts_private_bodies	Private note transparently readable on read-back	US-04, FR-04
Integration	test_list_notes_skips_undecryptable_private_note	Undecryptable note excluded + logged, not crashed/leaked	FR-07, SPR-02
Integration	test_create_private_without_privacy_service_raises	No key configured to refuse, never silent plaintext	SPR-01

The single most important test in this document is *test_create_private_note_encrypts_body_on_disk*. It asserts the literal **absence** of the plaintext string in the file on disk. That one assertion is the difference between 'we believe it is encrypted' and 'we proved it is.'

Test Levels - What Goes Where and Why

Unit (test_note, test_validation, test_privacy): one component, no I/O wiring. Validation rules and Fernet round-trips are pure logic - provable in milliseconds without a filesystem. These are the bulk of the suite because they are the cheapest place to catch a regression.

Integration (test_repository, test_note_manager): two or more tiers wired through the real dependency-injection seam, writing to a *tmp_path* directory. 'Encrypted on disk' and 'skip undecryptable note' only have meaning once the manager, privacy service, and file repository are connected, so they cannot be unit tests by definition.

Feature-level (test_cli, and the GUI controller test as the presentation tier lands): drives the app through its user-facing entry point with scripted input and a captured output buffer. It does not re-test encryption internals - it asserts the user-visible outcome. Intentionally thin because the presentation tier is intentionally thin.

I deliberately do **not** mock the filesystem in the integration tests. *tmp_path* gives a real directory destroyed after the test, faithful to production and still satisfying SPR-03 ('tests run without external dependencies'). A mocked filesystem would let a test pass while the real *json.dumps* / *Path.write_text* path is broken - exactly the false green this feature cannot afford.

When These Tests Run During Development

Test level	When it runs	Why then
Unit	On every save / before every commit (TDD inner loop)	Sub-second feedback; this is where you live while writing logic.
Integration	Before committing a slice, and in CI on every push	Slower (touches disk); run when a slice is done to prove the tiers still talk.
Feature-level	Before a user story is marked Done, and at each milestone gate	Slowest and broadest; the Definition-of-Done check, not the inner loop.

The shift-left point: the create-private security test is an *integration* test, but it runs in CI on every single push, not just at milestones. A security regression is the one thing I never want to discover late, so I move that specific test as far left as it will go.

What I Deliberately Did NOT Test First

- **NFR-01 / NFR-03 (performance)**: runtime assertions, not behavior. They belong in a Sprint 1 benchmark harness; testing them now would be guessing.
- **GUI widget rendering**: the presentation tier is thin by design and a broken button is loud and immediately visible. Pixel/widget tests are brittle and low-value this early; I test the controller seam instead.
- **Search (US-06)**: not implemented yet. Writing tests for unwritten code is theater.
- **FR-08 sort beyond the created_at tiebreaker**: basic ordering is covered; exhaustive ordering edge cases are not first-week risk.

AI Reflection

How AI helped. I used GitHub Copilot to brainstorm candidate cases for both features and to critique my draft set for gaps. Its most useful single suggestion was the negative security assertion - checking that the plaintext string is *absent* from the file, not just that some ciphertext is present. My first draft of *test_create_private_note_encrypts_body_on_disk* only asserted the body looked encrypted; the stronger 'plaintext must not appear anywhere in the file' assertion came from an AI critique and is now the most important line in the suite. It also flagged that I had no test for the no-key-configured path, which became *test_create_private_without_privacy_service_raises*.

What I kept. The absent-plaintext assertion, the missing no-key case, and a suggestion to parametrize the whitespace-title inputs instead of writing three near-duplicate tests.

What I changed. AI proposed one large end-to-end test that created a private note, listed it, edited it, and deleted it in a single function. I split it back into focused tests - a failure in a four-step test does not tell you which step broke, and edit/delete are different user stories (US-02, US-03) that should not be smuggled into the create/private first set.

What I rejected. AI suggested a test asserting duplicate tags are rejected. CLAUDE.md explicitly records duplicate tags as an *accepted* known limitation (they preserve user intent), so that test would encode the wrong behavior and fail a correct implementation. I also rejected mocking *Path.write_text* in the repository tests - that would produce a false green if the real serialization path regressed, unacceptable for the persistence and privacy features specifically. Keeping *tmp_path* over a mock is my call and the single most consequential testing judgment here.

The level assignments, the timing table, the out-of-scope list, and every keep/change/reject decision are my judgment. Copilot widened the candidate list; it did not decide what was worth testing.